Semi-simplicial types from a type theory of weakenings and thinnings

and why we always meet an obstruction

Thorsten Altenkirch

Draft, October 2026

Abstract

We explain an attempt to define semi-simplicial types (SST) in homotopy type theory by first defining a small type theory whose contexts are exactly the data of semi-simplicial types, constructing $\mathrm{SST}\,n$ uniformly in that syntax, and then interpreting the syntax in the universe. The syntax can be defined without any truncation and without K, and the construction of $\mathrm{SST}\,n$ goes through. The interpretation does not: it needs equations to hold definitionally which only hold propositionally, and patching this with transports needs uniqueness of identity proofs for the semantic types. Moving the problem into the syntax, by making composition a constructor, does not remove it either: it reappears as a coherence problem in the syntax. The Agda development accompanying this note is in the repository `github.com/txa/sst-new`.

1 Why semi-simplicial types?

Homotopy type theory describes ∞-groupoids: every type comes with its identity types, and J supplies all their coherences for free. It does not (yet) describe higher *categories*, where morphisms need not be invertible. The standard way to present an $(\infty, 1)$-category is by a (semi-)simplicial object of types satisfying Segal and completeness conditions, and Capriotti and Kraus showed that this works in a type theory where semi-simplicial types are available [10]. So if we could define the type of semi-simplicial types in HoTT, we could represent higher categories internally, including the $(\infty, 1)$-category of types itself.

A closely related goal is for HoTT to "eat itself": to define the syntax of HoTT inside HoTT and interpret it in the universe. Shulman's account of this problem [15] singles out semi-simplicial types as one of the most important open problems on the way, since they are the simplest structure that needs an infinite tower of coherences. (The title echoes Chapman's *Type theory should eat itself* [11].)

The question of defining semi-simplicial types in HoTT was raised during the special year on Univalent Foundations at the IAS in 2012–13, by Voevodsky and others [16]. Voevodsky's proposal of a type system with two identity types, one strict and one homotopical [17], makes it possible, and this idea was developed into two-level type theory [1, 9], in which semi-simplicial types can be defined. Herbelin gave a construction [12] which, as he later pointed out, relies on uniqueness of identity proofs. Kolomatskaia and Shulman define them in displayed type theory [13]. In book HoTT the problem remains open.

2 Semi-simplicial types up to triangles

A semi-simplicial type up to level 2 consists of

$$X_0 : \mathcal{U}$$
$$X_1 : X_0 \to X_0 \to \mathcal{U}$$
$$X_2 : (x_0\, x_1\, x_2 : X_0) \to X_1\, x_0\, x_1 \to X_1\, x_0\, x_2 \to X_1\, x_1\, x_2 \to \mathcal{U}$$

Points, edges between points, and triangles filling a boundary of three points and three edges. The problem is to define, for a variable $n : \mathbb{N}$, the type of such data up to level n. For each numeral n we can write it down, but nobody knows how to do it uniformly in n in book HoTT.

Two observations about the example guide everything that follows.

1. Each X_k is a family over a *telescope* of arguments, and each entry of that telescope is an earlier family applied to a *selection* of earlier entries. The edge $x_{02} : X_1\, x_0\, x_2$ selects x_0 and x_2 from the three points.

2. The selections have to be compatible. The arguments of X_1 are two points, so the selection that picks out x_{02} must also pick out the two points it connects. Selections compose, and at level 2 this composition is used for the first time in a non-trivial way: the boundary of a triangle consists of edges, and the boundary of an edge consists of points.

Level 2 is therefore the smallest example in which everything below happens.

3 A type theory with only weakenings and thinnings

We describe the data above by a small type theory with five sorts:

`Con`	contexts	sequences of type families $X_0, X_1, \ldots$
`Ty Γ`	types	telescopes (domains of families)
`Var Γ A`	variables	a family in Γ over the telescope A
`Wk Δ Γ`	weakenings	forgetting the last families of a context
`Thin A B`	thinnings	projections from telescope A to B

There are two directions of movement:

- *vertically*, a weakening goes from a longer to a shorter context: one more family is in scope. We only need one step, `p : Wk (Γ ▷ A) Γ`, and no general substitution;
- *horizontally*, within a fixed context, a thinning goes from a longer to a shorter telescope by dropping some entries and keeping others.

The constructors are

```
•    : Con
_▷_  : (Γ : Con) → Ty Γ → Con          -- add a family over a telescope
ε    : Ty Γ                            -- empty telescope
ext  : (A : Ty Γ) → Var Γ B → Thin A B → Ty Γ
                                       -- A extended by an entry  x (δ a)
zero : Var (Γ ▷ A) (wkTy A p)
suc  : Var Γ A → Var (Γ ▷ B) (wkTy A p)
tε   : Thin ε ε
t-d  : Thin A B → Thin (ext A x δ) B   -- drop the last entry
t-k  : Thin A B → Thin (ext A x ?) (ext B x δ)   -- keep it
```

The variables `zero` and `suc` need weakening: `zero` refers to the last family of `Γ ▷ A`, whose telescope `A` lives in the shorter context `Γ`, so its type is `A` moved along `p`. Weakening is not a constructor but an operation, defined by recursion on the syntax, which pushes the weakening through types, variables and thinnings:

```
wkTy   : Ty Γ → Wk Δ Γ → Ty Δ
wkVar  : Var Γ A → (w : Wk Δ Γ) → Var Δ (wkTy A w)
wkThin : Thin A B → (w : Wk Δ Γ) → Thin (wkTy A w) (wkTy B w)

wkTy ε           w = ε
wkTy (ext A x δ) w = ext (wkTy A w) (wkVar x w) (wkThin δ w)
wkVar x p          = suc x
wkThin tε      w   = tε
wkThin (t-d σ) w   = t-d (wkThin σ w)
wkThin (t-k σ) w   = t-k (wkThin σ w)
```

Weakening a telescope only renames the families it refers to (each x becomes `suc x`); the shape of the telescope and of its thinnings is unchanged. For example the edges E_{ij} above have target `wkTy A₁ p`: the two-point telescope A_1 moved into the context where X_1 is in scope. In the intended interpretation a context is a nested Σ-type of families, a telescope A is a type depending on the context, `ext A x δ` is $\Sigma\,(a : A)\,.\,x\,(\delta\,a)$, and a thinning is a projection.

In this language the example reads:

```
Γ₀ = • ▷ ε                                          -- X₀
A₁ = ext (ext ε zero tε) zero (t-d tε)              -- two points
Γ₁ = Γ₀ ▷ A₁                                        -- X₁
V₃ = three points (variable suc zero)
E₀₁ E₀₂ E₁₂ : Thin V₃ (wkTy A₁ p)                   -- the three edges
A₂ = ext (ext (ext V₃ zero E₀₁) zero (t-d E₀₂)) zero (t-d (t-d E₁₂))
```

The thinnings E_{ij} select the endpoints of the three edges, for example `E₁₂ = t-k (t-k (t-d tε))` drops x_0 and keeps x_1, x_2.

What is the ?? If we keep the last entry $x\,(\delta\,b)$ of `ext B x δ` while thinning the rest along σ : `Thin` $A\,B$, the entry of the source must be x applied to the composite selection: the index is $\delta \circ \sigma$.

```
t-k : (σ : Thin A B) → Thin (ext A x (δ ∘ σ)) (ext B x δ)
```

This is where composition enters. In `E₁₂`, keeping x_2 requires x_2's stored selection in V_3 (drop x_0 and x_1) to be the composite of the selection in A_1 (drop the first point) with the thinning that drops x_0.

4 The syntax: induction-recursion

We want the syntax to be an ordinary inductive definition, *not* a quotient. A quotient inductive-inductive type is a set, and its recursor only eliminates into sets, whereas we want to interpret into the universe, which is not a set. So the syntax must be untruncated: equality of syntax is the equality of an inductive type, and it is a theorem (not an axiom) that it is a set.

The type of `t-k` mentions `δ ∘ σ`, and the types of `zero` and `suc` mention `wkTy`. So composition and weakening have to be defined *simultaneously* with the syntax: the syntax is an inductive-inductive-recursive definition, in which functions defined by recursion on the types being defined

occur in the indices of their constructors. (In the code we package the sorts as one family `El : U → Set` indexed by codes; this is only a convenient way to write the mutual definition in Agda, not a reduction to something simpler.)

We should be honest about the status of this. Agda accepts such definitions, but they are a feature of Agda whose semantics is not well understood. There are established accounts of induction-recursion [2, 3] and of induction-induction [7], but none that we know of covers definitions like ours, where recursively defined functions, and in the later versions also proofs about them, appear in constructor indices. We use it anyway, as a powerful piece of "magic" that lets us state and check the constructions. The point of the rest of this note is that even with this magic we still run into an obstruction.

First attempt: index by the composite. With `t-k` as above, define composition by recursion on the right argument:

```
δ      ∘ tε    = δ
δ      ∘ t-d σ = t-d (δ ∘ σ)
t-d δ  ∘ t-k σ = t-d (δ ∘ σ)
t-k δ  ∘ t-k σ = t-k (δ ∘ σ)          -- ill-typed!
```

The left side has type `Thin (ext A x ((δC ∘ δ) ∘ σ)) _`, the right side `Thin (ext A x (δC ∘ (δ ∘ σ))) _`. They agree only up to associativity, a theorem we prove by induction, so we would have to write `subst _ ∘-assoc (t-k (δ ∘ σ))`. The same happens for the identity (`t-k id` has index `δ ∘ id`, needing `∘-id`) and for weakening a thinning (needing `wkThin (δ ∘ σ) ≡ wkThin δ ∘ wkThin σ`). A `subst` along a propositional equation in a definition blocks computation in everything that uses it. In addition, pattern matching on `t-k` forces Agda to unify an index with `δ ∘ σ`, a defined function ("green slime", in McBride's terminology).

Second attempt: Fording. Following McBride, we store the equation instead of computing the index:[1]

```
t-k : (σ : Thin A B) → δ ∘ σ ≡ δ' → Thin (ext A x δ') (ext B x δ)
```

Now composition is defined without any transport; the equation is built as data:

```
t-k δ q ∘ t-k σ e = t-k (δ ∘ σ) (∘-k q e)
  where ∘-k : ρ ∘ δ ≡ δB → δB ∘ σ ≡ δA → ρ ∘ (δ ∘ σ) ≡ δA
```

and `∘-k` uses `∘-assoc`, which is proved mutually with `_∘_`. Identity and weakening are handled the same way. Closed terms compute: the example above is typechecked with all stored equations being `refl`, and `id ∘ id ≡ id` holds by `refl` at A_2.

The new problem: UIP in the syntax. To prove the laws (associativity, identity) we need a congruence for `t-k`:

```
t-k-cong : σ ≡ σ' → t-k σ e ≡ t-k σ' e'
```

where `e` and `e'` are two proofs of the same equation `δ ∘ σ ≡ δ'`. This needs them to be equal, i.e. uniqueness of identity proofs for `Thin`. In Agda this is K (directory `withK/`).

[1] The name alludes to Henry Ford's slogan that a customer can have a car painted any colour, so long as it is black. Instead of restricting the index of `t-k` to be `δ ∘ σ`, we allow any index `δ'`, so long as it is equal to `δ ∘ σ`. The name is due to Conor McBride, who speaks of reformulating an indexed family "in the Henry Ford tradition" [19, Chapter 6]; the encoding of indices by equations is also used in [18]. We have not found an earlier written source; the phrase may have been used in talks before.

5 Removing UIP: codes for equality

The syntax is finitary and first order, so all its sorts have decidable equality, and by Hedberg's theorem [4] they are sets. Rather than going through Hedberg we encode equality of thinnings directly, as a type defined by recursion (the encode-decode method [6], [5, §8.1]):

```
Code : Thin A C → Thin A' C' → Set
Code tε          tε          = ⊤
Code (t-d σ)     (t-d τ)     = Code σ τ
Code (t-k σ _ _) (t-k τ _ _) = Code σ τ
Code _           _           = ⊥

t-k : (σ : Thin A B) → CodeV x y → Code (δ ∘ σ) δ'
    → Thin (ext A x δ') (ext B y δ)
```

- Codes are propositions by a plain induction (`Code-prop`): no K, no Hedberg, no `Prop`.
- The kept variable is split into `x` and `y`, related by a code `CodeV x y`, so that no index is repeated and pattern matching works `--without-K`.
- Codes can be decoded into equations (`Code-sound`), and the total space of variables is a set (`VarΣ-set`).

With this, `t-k-cong` holds without K. This achieves the first goal: an *untruncated* syntax, defined by induction-recursion, whose sorts are provably sets, all `--without-K` (file `TT.agda`).

6 Semi-simplicial types in the syntax

Following Section 3 of [1], we define by structural recursion on k, simultaneously,

- the context `ctx k` of levels $X_0, \ldots, X_{k-1}$,
- the k-skeleton `SK k n : Ty (ctx k)` of an $(n-1)$-simplex,
- its face maps `SK→ k f : Thin (SK k n) (SK k m)` for order preserving embeddings f,
- their functoriality `α k f g : SK→ k g ∘ SK→ k f ≡ SK→ k (f ∘ g)`.

Level $k+1$ only uses level k weakened once, so all types agree definitionally. The transport along α_k in the paper becomes the equation stored in `t-k`, and the coherence of α, for which the paper used K, becomes `t-k-cong`, which is now K-free. The result is

```
SST : ℕ → Con
```

defined uniformly in n, without K (file `SSTk.agda`).

7 The interpretation, and the obstruction

To finish we need the standard interpretation of the syntax in the universe:

```
⟦ Γ ⟧C  : Set₁                          -- nested Σ of families
⟦ A ⟧T  : ⟦ Γ ⟧C → Set
⟦ x ⟧V  : (γ : ⟦ Γ ⟧C) → ⟦ A ⟧T γ → Set
⟦ σ ⟧Th : ⟦ A ⟧T γ → ⟦ B ⟧T γ           -- projections

⟦ ext A x δ ⟧T γ = Σ (⟦ A ⟧T γ) (λ a → ⟦ x ⟧V γ (⟦ δ ⟧Th a))
```

Then ⟦ SST n ⟧C would be the type of semi-simplicial types up to level n.

Where it breaks. Interpreting a kept entry needs the interpretation of the stored equation:

```
⟦ t-k σ e ⟧Th (a , v) = ⟦ σ ⟧Th a , subst ⟦ x ⟧V (keq e a) v
  where keq e a : ⟦ δ' ⟧Th a ≡ ⟦ δ ⟧Th (⟦ σ ⟧Th a)
```

and `keq` is built from

```
⟦∘⟧ : ⟦ δ ∘ σ ⟧Th a ≡ ⟦ δ ⟧Th (⟦ σ ⟧Th a)
```

which we can only prove by induction: it is a propositional, not a definitional, equality. In the case `t-k ∘ t-k` of ⟦∘⟧ itself, the element v is transported along two *different* paths, one coming from `∘-k q e` and one from composing the two stored equations, and we have to show that the results agree. Those paths live in the semantic types ⟦ C ⟧T γ, which are arbitrary types, so we need uniqueness of identity proofs for them (the lemma `pair-irr`, equivalent to UIP). Weakening causes the same problem: ⟦ wkTy A p ⟧T (γ , X) is related to ⟦ A ⟧T γ only by a map whose naturality (`unwk-nat`) holds propositionally.

Put differently, the interpretation would go through if the equations

$$\llbracket \delta \circ \sigma \rrbracket = \llbracket \delta \rrbracket \circ \llbracket \sigma \rrbracket \qquad \text{and} \qquad \llbracket \mathrm{wk}\, A \rrbracket(\gamma, X) = \llbracket A \rrbracket\, \gamma$$

held definitionally. They do hold for closed terms: for every numeral n, $\llbracket \mathrm{SST}\, n \rrbracket$ computes to the expected iterated Σ-type, and Agda checks this by `refl`. They do not hold for a *variable* n, and uniformity in n is the whole point.

In Agda the interpretation `Sem.agda` is K-free except for one module parameter, `pair-irr`, which is proved with K in `SemK.agda`. So we still do not have a semantic encoding of SST in HoTT.

Remark. *In two-level type theory the equations can be taken as* outer *(strict) equalities, which satisfy UIP. Read this way, the K-based interpretation is essentially the known construction of semi-simplicial types in 2LTT [1, 9]. The open problem is precisely to get rid of the outer level.*

8 An alternative: composition as a constructor

The obstruction comes from composition being a defined function whose interpretation is only propositionally functorial. We could instead make weakening and composition *constructors*, and turn their recursive equations into path constructors: a higher inductive-inductive type (file `HIIT.agda`, a variant of Kovács' optimised syntax, see [8, Appendix A]).

- The semantic problem disappears. In the standard model composition is function composition, and every path constructor is interpreted by `refl`, without K.
- The problem moves into the syntax. The keep rule now has to be stated as a path *over* associativity,

```
t-k∘t-k : subst (λ d → Thin (ext A x d) _) ass (t-k δ ∘ t-k σ) ≡ t-k (δ ∘ σ)
```

so `ass` is a path constructor, and it overlaps with the computation rules. For example, on `(ρ ∘ δ) ∘ tε` we can go via `ass` or via `∘tε`; the two paths form a loop that nothing fills. These critical pairs would need 2-dimensional path constructors (the pentagon), which create critical pairs of their own, and so on.

The interpretation of the HIIT into the inductive-recursive syntax gives a map HIIT $\to$ IR (file `HIITtoIR.agda`). The hope is that this map is an equivalence (`HIITConj.agda`): then the HIIT is a set, and the IR syntax inherits the K-free standard model. We have not carried this out. A sketch suggests that, as stated, it fails under univalence: in a model where `Thin A B` carries an extra circle component and `ass` is interpreted with a twist by `loop`, the loop above is sent to `loop`, which is not `refl`.

9 The obstruction is always there

The attempts can be summarised as follows.

presentation	syntax	standard model
IR, index by $\delta \circ \sigma$	transports block computation	—
IR, Forded (`withK/`)	needs UIP for `Thin`	needs UIP
IR, codes (`TT.agda`)	a set, K-free	needs UIP
HIIT (`HIIT.agda`)	critical pairs (`ass`)	all `refl`
HIIT, Forded and conditional	a set (expected)	needs UIP

Each version either puts the coherence of composition into the syntax, as critical pairs, or into the semantics, as UIP. What is never removed is the coherence of composing *non-invertible* maps, here thinnings. For invertible structure, identity types and J supply all coherence for free (for instance, coherently associative groups are exactly loop spaces). For one non-invertible level nothing analogous is known in book HoTT. The same obstruction appears in a minimal form for coherently associative semigroups (A_∞ without units), which suggests that a solution would have to add a genuinely directed principle, as two-level type theory [9], simplicial type theory [14] or displayed type theory [13] do.

Use of AI tools

This note and the accompanying Agda development were produced in collaboration with Claude (Anthropic), used through Claude Code. Claude drafted text and Agda code, carried out experiments in Agda, and searched the literature, under the author's direction. The author has checked the content and is responsible for it. Claims described as sketches have not been formally verified.

References

[1] Thorsten Altenkirch, Paolo Capriotti and Nicolai Kraus. Extending homotopy type theory with strict equality. In *CSL 2016*, LIPIcs 62, pages 21:1–21:17, 2016.

[2] Peter Dybjer. A general formulation of simultaneous inductive-recursive definitions in type theory. *Journal of Symbolic Logic*, 65(2):525–549, 2000.

[3] Peter Dybjer and Anton Setzer. A finite axiomatization of inductive-recursive definitions. In *TLCA 1999*, LNCS 1581, pages 129–146. Springer, 1999.

[4] Michael Hedberg. A coherence theorem for Martin-Löf's type theory. *Journal of Functional Programming*, 8(4):413–436, 1998.

[5] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013.

[6] Daniel R. Licata and Michael Shulman. Calculating the fundamental group of the circle in homotopy type theory. In *LICS 2013*, pages 223–232. IEEE, 2013.

[7] Fredrik Nordvall Forsberg and Anton Setzer. Inductive-inductive definitions. In *CSL 2010*, LNCS 6247, pages 454–468. Springer, 2010.

[8] Johann Rosain. A coherent syntax for intrinsic type theories. Internship report, Eötvös Loránd University, Budapest, 2026.

[9] Danil Annenkov, Paolo Capriotti, Nicolai Kraus and Christian Sattler. Two-level type theory and applications. *Mathematical Structures in Computer Science*, 33(8):688–743, 2023.

[10] Paolo Capriotti and Nicolai Kraus. Univalent higher categories via complete semi-Segal types. *Proceedings of the ACM on Programming Languages*, 2(POPL):44:1–44:29, 2018.

[11] James Chapman. Type theory should eat itself. *Electronic Notes in Theoretical Computer Science*, 228:21–36, 2009.

[12] Hugo Herbelin. A dependently-typed construction of semi-simplicial types. *Mathematical Structures in Computer Science*, 25(5):1116–1131, 2015.

[13] Astra Kolomatskaia and Michael Shulman. Displayed type theory and semi-simplicial types. *Mathematical Structures in Computer Science*, published online 2025. arXiv:2311.18781.

[14] Emily Riehl and Michael Shulman. A type theory for synthetic ∞-categories. *Higher Structures*, 1(1):147–224, 2017.

[15] Michael Shulman. Homotopy type theory should eat itself (but so far, it's too big to swallow). Blog post, https://homotopytypetheory.org/2014/03/03/hott-should-eat-itself/, 2014.

[16] Univalent Foundations special year, IAS. Semi-simplicial types (open problem page). https://ncatlab.org/ufias2012/show/Semi-simplicial+types, 2012–13.

[17] Vladimir Voevodsky. A simple type system with two identity types. Unpublished note, 2013.

[18] James Chapman, Pierre-Évariste Dagand, Conor McBride and Peter Morris. The gentle art of levitation. In *ICFP 2010*, pages 3–14. ACM, 2010.

[19] Conor McBride. *Dependently Typed Metaprogramming (in Agda)*. Lecture notes, 2013. https://github.com/pigworker/MetaprogAgda.